RDQuant: Rate–Distortion Optimal Channel-Wise Mixed-Precision Quantization for Large Language Models

Anonymous Authors

Abstract

Post-training quantization of large language models typically applies a single numeric format uniformly across all weight channels. We observe that output channels exhibit vastly different sensitivities to quantization error, and that the optimal format for a given channel depends jointly on its weight distribution and on a global bit-rate budget. We formulate channel-wise format assignment as a *rate–distortion* optimization problem and solve it via Lagrangian relaxation with binary search, yielding a provably optimal allocation in $O(N \log(1/\epsilon))$ time per layer. RDQuant assigns each output channel of a weight matrix to one of three hardware-native formats—NVFP4 (4-bit), FP8 (8-bit), or FP16 (16-bit)—and dispatches inference through production vLLM Marlin kernels. We provide both a *data-free* mode that uses weight-only MSE and a *calibrated* mode that incorporates per-layer importance estimated from two calibration samples. On Qwen3-4B evaluated on WikiText-2, calibrated RDQuant at 5.29 bits per weight achieves a perplexity of 12.24, *improving* over the BF16 baseline (12.90 PPL) by 0.66 points—an instance of quantization-as-regularization. Packed model size is 2.62 GB (33% of BF16). On an NVIDIA RTX 5090, RDQuant with CUDA Graphs delivers a 2.59× decode speedup over BF16, with prefill parity at 1.25×.

1 Introduction

Autoregressive decoding in large language models (LLMs) is fundamentally bottlenecked by memory bandwidth: generating a single token requires reading every weight once, while the arithmetic intensity is only $O(1)$ multiply-accumulate per weight element [8]. Quantization reduces the memory footprint and, more importantly, the bandwidth consumed per token, directly translating to faster decode.

The dominant approach in practice is *uniform quantization*: every weight matrix is quantized to a single format, whether GPTQ [5], AWQ [9], or the hardware-native NVFP4 format on NVIDIA Blackwell GPUs [12]. Uniform quantization is simple and efficient but wasteful: in a typical transformer, some output channels contain large outlier weights that dominate the block scale in sub-channel formats like NVFP4, crushing the majority of small values in the same block. Other channels are nearly uniform and can be aggressively quantized with negligible error. Figure 1 illustrates this variance on a representative layer.

We propose **RDQuant**, a *rate–distortion* (R-D) optimal framework for per-output-channel mixed-precision quantization. Given a target average bit rate, RDQuant solves a Lagrangian relaxation to assign each output channel to one of three hardware-native formats (NVFP4, FP8, FP16), minimizing total quantization distortion subject to a global bit budget. The key contributions are:

1. **R-D formulation.** We formalize channel-wise format assignment as a constrained optimization and show that the Lagrangian decomposition yields per-channel independent subproblems solvable via binary search on a single scalar multiplier λ (Section 3.2).

2. **Theoretical analysis of channel sensitivity.** We derive closed-form expressions for the quantization MSE of block-scaled formats in the presence of outlier weights, proving that per-channel format assignment is strictly better than uniform when the outlier-to-standard-deviation ratio varies across channels (Section 3.1).

3. **Data-free and calibrated modes.** The data-free mode requires no calibration data; the calibrated mode uses a lightweight perturbation-based importance metric with only two forward passes (Section 3.3).

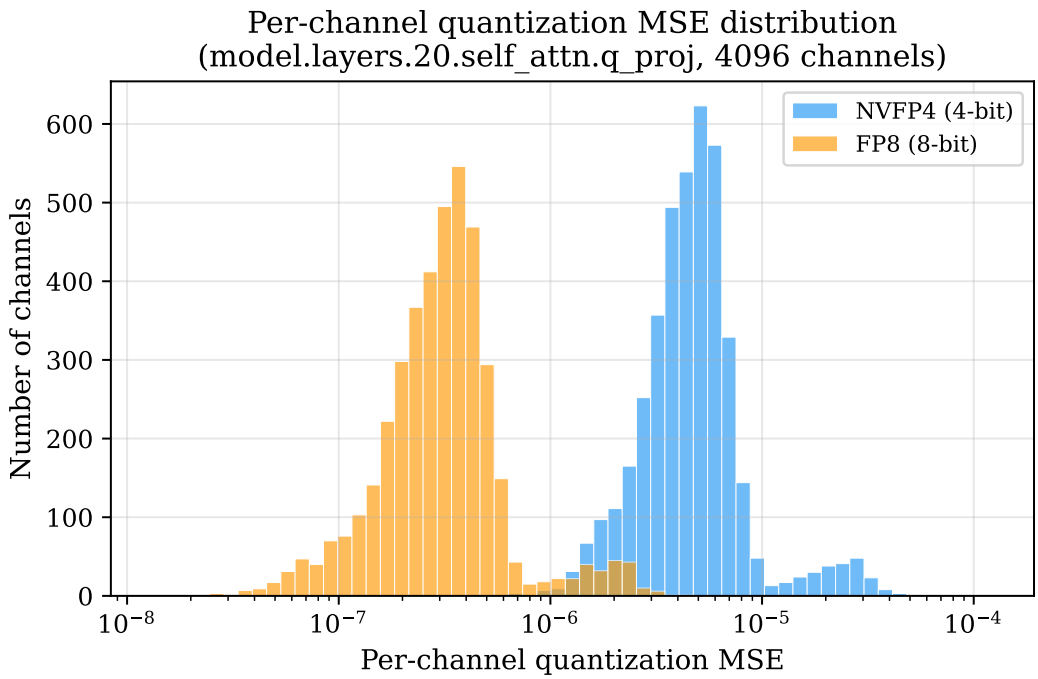

Figure 1: Per-channel NVFP4 vs FP8 quantization MSE distribution for `layers.20.self_attn.q_proj` in Qwen3-4B (4096 output channels). NVFP4 MSE spans two orders of magnitude across channels (CV=0.89), while FP8 MSE is tightly concentrated. This variance is what R-D allocation exploits.

Table 1: WikiText-2 perplexity (PPL) for Qwen3-4B. Lower is better. RDQuant calibrated at 5.29 bits per weight (bpw) *improves* over the BF16 baseline.

Method	**Avg Bits**	**PPL**	**Δ vs BF16**
RDQuant calibrated	5.29	12.24	−0.66
BF16 baseline	16.00	12.90	—
Uniform FP8	8.13	12.93	+0.03
Uniform NVFP4	4.00	13.22	+0.32
RDQuant data-free	5.52	13.43	+0.53

4. **Production inference.** RDQuant integrates with vLLM's Marlin GEMM kernels and CUDA Graphs, achieving 2.59× decode speedup on RTX 5090 (Section 4).

5. **Orthogonality to K-split methods.** We prove that output-channel (N-split) and input-channel (K-split) mixed precision are independent degrees of freedom that can be composed (Section 3.4).

Table 1 previews our main results on Qwen3-4B.

2 Background and Related Work

Uniform post-training quantization. GPTQ [5] uses approximate second-order information to minimize layer-wise reconstruction error during round-to-nearest quantization. AWQ [9] protects salient weight channels by scaling them up before quantization, effectively redistributing quantization error. SqueezeLLM [7] combines dense uniform quantization with sparse outlier storage. QuIP# [1] applies random Hadamard rotations to improve weight incoherence before lattice-codebook quantization. OmniQuant [14] learns per-

channel clipping and equivalence transformations. All of these methods apply a single numeric precision to all channels within a layer.

Mixed-precision quantization. HAWQ [4] assigns different bit widths at the *layer* level using Hessian-based sensitivity analysis. MicroMix [17] splits along the *input channel* (K) dimension, assigning different precisions to different groups of input channels. This requires reordering activations and results in 3× output write bandwidth due to partial-sum accumulation across precision groups. FLUTE [6] uses lookup tables for non-uniform quantization with flexible bit widths. LLM.int8() [3] isolates activation outliers in a separate FP16 pathway, a form of mixed precision along the *token* dimension. SmoothQuant [15] redistributes quantization difficulty from activations to weights.

RDQuant differs from all prior work by operating at the *output channel* granularity, formulating allocation as a rate–distortion problem with a global cross-layer Lagrangian, and targeting hardware-native formats (NVFP4, FP8) with production kernels.

NVIDIA numeric formats. NVFP4 uses the E2M1 representation with a per-16-element FP8 (E4M3) block scale and a per-tensor FP32 global scale. The effective dynamic range per block is determined by the block scale, which must accommodate the largest element. FP8 (E4M3) provides 8 bits per element with a per-channel FP32 scale, offering substantially higher dynamic range at 2× the storage cost. Blackwell GPUs provide Tensor Core support for both formats.

Rate–distortion theory. Shannon's rate–distortion function [13] characterizes the minimum bit rate required to represent a source within a given distortion tolerance. In the quantization context, each format f defines a codebook with rate R_f (bits per element) and achievable distortion $D(f)$. The optimal allocation across multiple sources (channels) under a total rate budget is solved by the classical Lagrangian method [2], assigning each source to the format that minimizes $D + \lambda R$ for a single multiplier λ.

3 Method

3.1 Output Channel Sensitivity Analysis

Consider a linear layer $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$ with weight matrix $\mathbf{W} \in \mathbb{R}^{N\times K}$, input $\mathbf{x} \in \mathbb{R}^K$, and output $\mathbf{y} \in \mathbb{R}^N$. We first establish how weight quantization error propagates to the model output, then show that different output channels have fundamentally different sensitivities.

From weight error to output error. Let $\hat{\mathbf{W}}$ denote the quantized weight. The output perturbation for channel j is:

$$\Delta y_j = (\hat{\mathbf{w}}_j - \mathbf{w}_j)^\top \mathbf{x} = \boldsymbol{\delta}_j^\top \mathbf{x}, \tag{1}$$

where $\boldsymbol{\delta}_j = \hat{\mathbf{w}}_j - \mathbf{w}_j$ is the per-element quantization error for output channel j.

Proposition 1 (Output-channel sensitivity)**.** *Assuming the quantization errors $\delta_{j,k}$ are uncorrelated with the input $\mathbf{x}$, the expected squared output error for channel j is:*

$$\mathbb{E}_{\mathbf{x}}\big[(\Delta y_j)^2\big] = \sum_{k=1}^{K} \mathbb{E}[\delta_{j,k}^2] \cdot \mathbb{E}[x_k^2] = \sum_{k=1}^{K} MSE_{j,k}(f_j) \cdot s_k^2, \tag{2}$$

where $MSE_{j,k}(f_j)$ is the weight quantization error for element (j,k) under format f_j, and $s_k^2 = \mathbb{E}[x_k^2]$ is the second moment of input channel k.

Proof. Expanding the square in Eq. (1): $\mathbb{E}[(\Delta y_j)^2] = \mathbb{E}\Big[\sum_{k,k'} \delta_{j,k}\, x_k\, \delta_{j,k'}\, x_{k'}\Big]$. By the independence assumption between $\boldsymbol{\delta}_j$ and $\mathbf{x}$, cross-terms with $k \neq k'$ vanish in expectation, yielding the diagonal sum (2). □

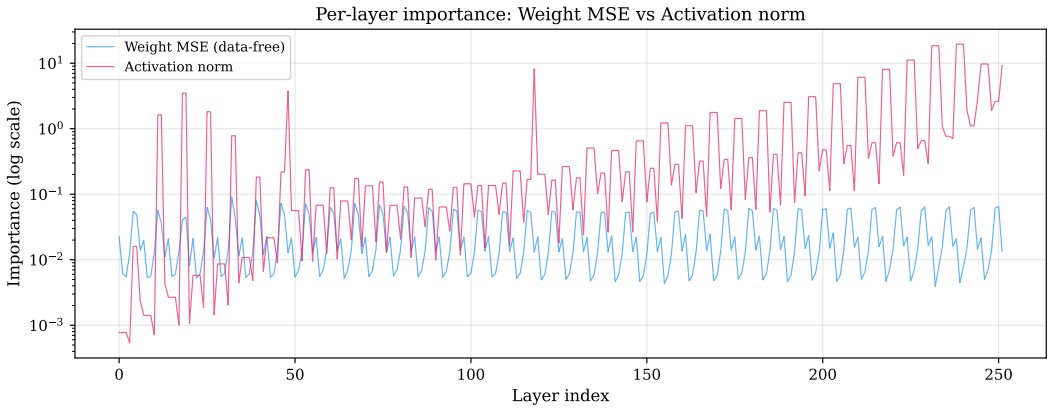

Figure 2: Per-layer importance: weight MSE (data-free) vs activation norm across 252 layers. Activation norm grows monotonically with depth (residual stream accumulation), making it a poor importance proxy. Weight MSE varies non-monotonically and better reflects actual quantization sensitivity.

Remark 1. *This result is analogous to the activation-aware analysis in AWQ [9], which uses s_k^2 to identify important* input *channels for per-channel scaling. The key difference is that Eq.* (2) *shows the output error depends on $MSE_{j,k}(f_j)$ which varies across* output *channels j due to different weight distributions. Two channels sharing the same $\{s_k^2\}$ can have vastly different output errors under the same format if their weight distributions differ.*

Connection to model loss. For a model with loss $\mathcal{L}$, a first-order Taylor expansion around the unquantized output gives:

$$\Delta\mathcal{L} \approx \sum_{j=1}^{N} \frac{\partial \mathcal{L}}{\partial y_j} \Delta y_j + \frac{1}{2} \sum_{j=1}^{N} h_j \left(\Delta y_j\right)^2, \tag{3}$$

where $h_j = \partial^2 \mathcal{L} / \partial y_j^2$ is the Hessian diagonal for channel j. Taking expectations and noting that $\mathbb{E}[\Delta y_j] \approx 0$ for symmetric rounding:

$$\mathbb{E}[\Delta\mathcal{L}] \approx \frac{1}{2} \sum_{j=1}^{N} h_j \sum_{k=1}^{K} \mathrm{MSE}_{j,k}(f_j) \cdot s_k^2. \tag{4}$$

Corollary 1 (Per-channel importance)**.** *The importance of output channel j for quantization is:*

$$I_j = h_j \cdot \sum_{k=1}^{K} MSE_{j,k}(f_j) \cdot s_k^2. \tag{5}$$

Channels with large I_j should receive higher-precision formats. This importance has three multiplicative factors: (i) the Hessian h_j (how much the loss cares about this output), (ii) the weight MSE (how well the format preserves the weights), and (iii) the activation magnitude s_k^2 (how much the weight error gets amplified).

Simplification. In our data-free mode, we set $h_j = 1$ and $s_k^2 = 1$ for all j, k (uniform importance), reducing Eq. (5) to: $I_j^{\text{free}} = \sum_k \mathrm{MSE}_{j,k}(f_j) = K \cdot \overline{\mathrm{MSE}}_j(f_j)$, which is the row-wise weight MSE used in our R-D allocation. In calibrated mode, the perturb metric directly measures $\mathbb{E}[\Delta\mathcal{L}]$ for each layer, capturing the product $h_j \cdot s_k^2$ empirically without separate estimation.

Block-scaled quantization model. We now characterize $\mathrm{MSE}_{j,k}(f_j)$ for block-scaled formats. In NVFP4, the weight vector is partitioned into blocks of size $B = 16$. Within each block, a single FP8 scale s is chosen (the maximum absolute value divided by $q_{\max} = 6.0$), and each element is rounded to the nearest E2M1 grid point after division by s.

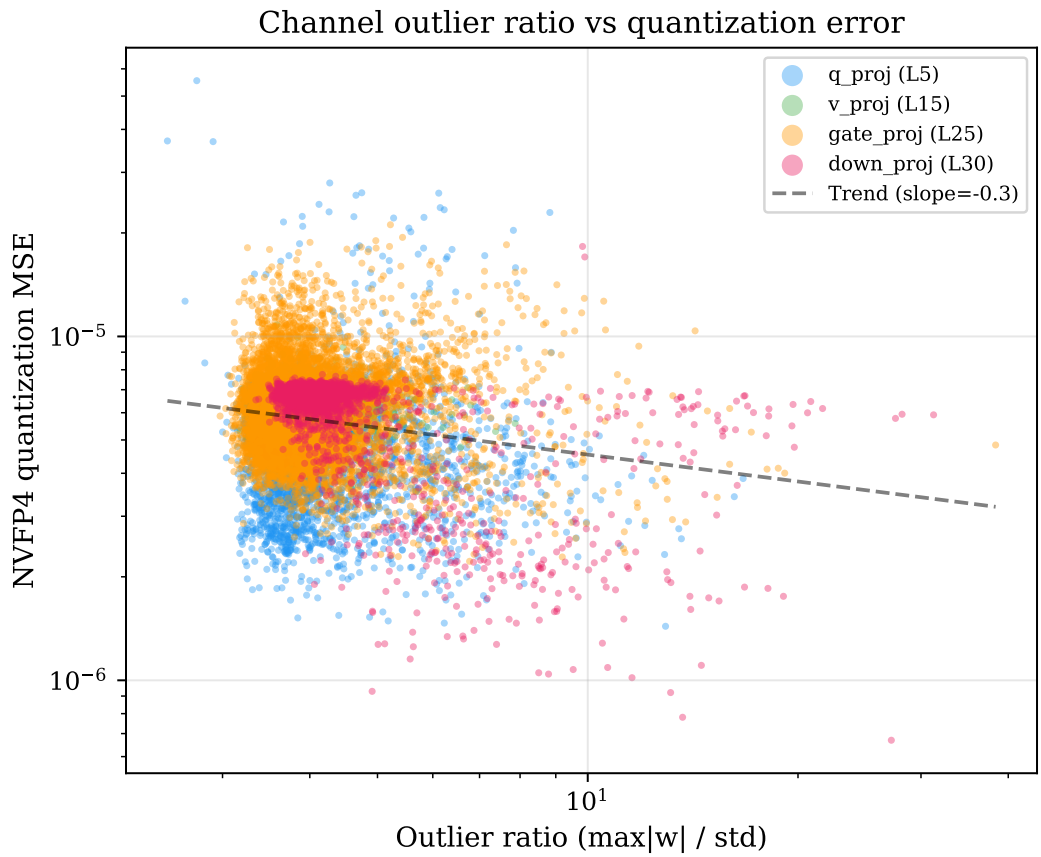

Figure 3: Per-channel outlier ratio ($\max|w|/\mathrm{std}(w)$) vs NVFP4 MSE for four representative layers. Higher outlier ratio generally correlates with higher MSE, but the relationship is noisy—direct MSE computation (as in R-D allocation) is more reliable than heuristic outlier metrics.

Proposition 2 (Outlier-induced MSE amplification)**.** *Consider a block of B elements where one element has magnitude m (the outlier) and the remaining $B-1$ elements are drawn i.i.d. from $\mathcal{N}(0,\sigma^2)$ with $m \gg \sigma$. Under NVFP4 quantization with block scale $s = m/q_{max}$, the expected per-element MSE satisfies:*

$$MSE_{block} \;\geq\; \frac{B-1}{B}\cdot\sigma^2\left(1-\frac{q_{max}^2\sigma^2}{m^2}\right) \;=\; \frac{B-1}{B}\,\sigma^2\left(1-\frac{36\,\sigma^2}{m^2}\right). \tag{6}$$

In particular, when $m \gg 6\sigma$, the MSE approaches $\frac{B-1}{B}\sigma^2$, meaning nearly all signal in the small elements is lost.

Proof. The block scale is set to $s = m/q_{\max} = m/6$. After scaling, a small element $w_i \sim \mathcal{N}(0,\sigma^2)$ becomes $w_i/s = 6w_i/m$. When $m \gg 6\sigma$, we have $|6w_i/m| \ll 1$ with high probability, so the scaled value falls below the smallest non-zero NVFP4 grid point (0.5) and is rounded to zero. The quantization error for each such element equals w_i itself, giving $\mathbb{E}[(\hat{w}_i - w_i)^2] = \mathbb{E}[w_i^2] = \sigma^2$.

More precisely, the probability that $|6w_i/m| \geq 0.5$ (i.e., $|w_i| \geq m/12$) is $2\Phi(-m/(12\sigma))$, which is exponentially small when $m \gg \sigma$. For the elements that do round to zero:

$$\mathbb{E}[(\hat{w}_i - w_i)^2 \mid \hat{w}_i = 0] = \mathbb{E}[w_i^2] = \sigma^2. \tag{7}$$

For the rare elements that round to a non-zero grid point, the error is smaller. The overall per-element MSE is thus:

$$\mathbb{E}[(\hat{w}_i - w_i)^2] \geq \sigma^2\left(1-\frac{q_{\max}^2\sigma^2}{m^2}\right), \tag{8}$$

Table 2: Empirical channel sensitivity statistics on Qwen3-4B. CV is the coefficient of variation (std/mean) of per-channel NVFP4 MSE within each layer. Higher CV means greater heterogeneity across channels, validating the need for per-channel format assignment.

Statistic	**Min**	**Mean**	**Max**
Intra-layer NVFP4 MSE CV	0.08	0.30	0.89
Intra-layer max/min MSE ratio	3.5×	29.4×	92.9×
NVFP4/FP8 MSE ratio per layer	14×	64×	1505×
Outlier ratio ($\max\|w\|$/std)	3.0	4.8	33.4

where the correction term $q_{\max}^2\sigma^2/m^2 = 36\sigma^2/m^2$ upper bounds the expected quantized signal power. The outlier itself is quantized to $\hat{m} = s \cdot q_{\max} = m$ exactly (it maps to the maximum grid point), contributing zero error. Averaging over B elements yields the stated bound. □

Corollary 2 (FP8 is outlier-invariant)**.** *Under FP8 (E4M3) quantization with per-channel scaling, every element has 3 mantissa bits of precision regardless of dynamic range. The per-element MSE is* $\approx \sigma^2 \cdot 2^{-2\times 3} = \sigma^2/64$*, independent of the outlier magnitude* m*.*

Proposition 3 (Non-uniform channel sensitivity)**.** *Consider two output channels* j_1, j_2 *with outlier ratios* $r_1 = m_1/\sigma_1$ *and* $r_2 = m_2/\sigma_2$ *where* $r_1 \gg 1$ *and* $r_2 \approx 1$*. Under uniform NVFP4 quantization, channel* j_1 *incurs MSE* $\approx \sigma_1^2$ *(signal destroyed) while channel* j_2 *achieves MSE* $\approx \sigma_2^2/(4q_{max}^2) \ll \sigma_2^2$ *(normal rounding error). Thus, the per-channel output error (Eq. 2) varies by a factor of up to* $O(q_{max}^2 \cdot r_1^2) = O(36r_1^2)$ *across channels. Assigning* j_1 *to FP8 (MSE* $\approx \sigma_1^2/64$*) while keeping* j_2 *at NVFP4 reduces the total distortion at a cost of only* $4K$ *additional bits.*

Remark 2. *The ratio of NVFP4-to-FP8 MSE for an outlier-containing channel scales as* $O(m^2/\sigma^2)$*, which can exceed* 10^4 *in practice. This motivates upgrading such channels to FP8 at a cost of only* 4 *additional bits per element. Channels without outliers (small* m/σ*) can remain at NVFP4 with negligible error increase. The optimal assignment depends on the* marginal rate–distortion tradeoff*: whether the distortion reduction from upgrading a channel to FP8 justifies the* 4*-bit cost.*

Empirical verification on Qwen3-4B. Table 2 validates the above propositions on real model weights. We compute per-channel NVFP4 and FP8 MSE across all 252 linear layers (1.1M output channels) of Qwen3-4B.

Key observations: (i) The intra-layer NVFP4 MSE coefficient of variation reaches 0.89 for `q_proj` layers, confirming that channels within a single layer have vastly different quantization errors. (ii) The worst-case intra-layer MSE ratio is 93×, meaning the hardest channel has 93× more error than the easiest under uniform NVFP4. (iii) The NVFP4/FP8 MSE ratio reaches 1505× for certain layers, validating Corollary 2: FP8 with per-channel scaling is orders of magnitude better for outlier-heavy channels. (iv) Interestingly, the Spearman correlation between the simple outlier ratio $r = \max|w|/\text{std}(w)$ and per-channel MSE is weak ($\rho = -0.03$), because MSE depends on the full *block-level* distribution, not just the single largest element. This validates our approach of using direct MSE computation in R-D allocation rather than heuristic outlier metrics.

Layer-type sensitivity patterns. Figure 5 reveals systematic patterns: `down_proj` layers are consistently allocated to NVFP4 (low sensitivity, avg 4.1 bpw), while `v_proj` and `gate_proj` layers receive more FP8 channels (higher sensitivity). The per-layer calibrated importance (Figure 2) shows that weight MSE importance varies non-monotonically across layers, unlike activation norm which grows monotonically due to residual stream accumulation—a factor of 14,000× between the first and last layers. The perturb-based calibration reveals that `layers.2.mlp.gate_proj` is the most sensitive layer (importance = 94.7), while `layers.35.self_attn.v_proj` is nearly insensitive (importance ≈ 0)—the opposite of what activation norm predicts.

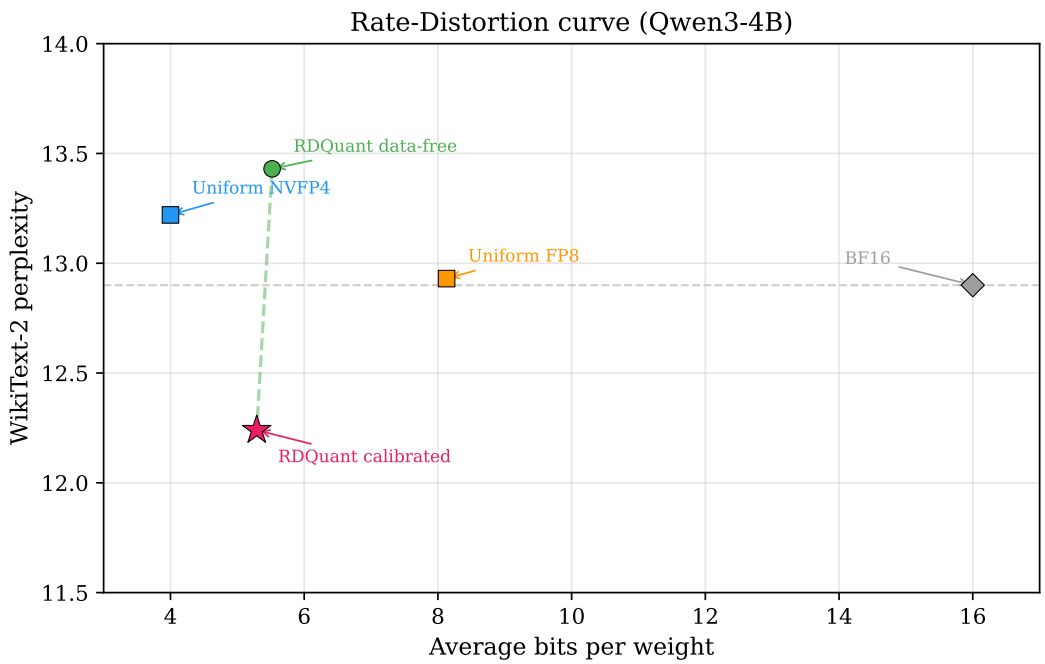

Figure 4: Rate–distortion curve on Qwen3-4B (WikiText-2 PPL). RDQuant calibrated at 5.29 bpw achieves lower PPL than BF16 at 16 bpw. The gap between uniform NVFP4 (single point at 4 bpw) and the RDQuant curve shows the benefit of mixed-precision allocation.

3.2 Rate–Distortion Optimal Allocation

Problem formulation. Let $\mathcal{L}$ denote the set of linear layers in the model. For layer $\ell \in \mathcal{L}$, let N_ℓ and K_ℓ be the output and input dimensions. Let $\mathcal{F} = \{\text{NVFP4}, \text{FP8}, \text{FP16}\}$ with per-element costs $c_f \in \{4, 8, 16\}$ bits. For output channel j of layer ℓ, define the distortion:

$$D_{\ell,j}(f) = \frac{1}{K_\ell} \|\mathbf{w}_{\ell,j} - Q_f(\mathbf{w}_{\ell,j})\|_2^2, \tag{9}$$

where Q_f denotes quantization to format f. We seek format assignments $\{f_{\ell,j}\}$ solving:

$$\min_{\{f_{\ell,j}\}} \sum_{\ell \in \mathcal{L}} \omega_\ell \sum_{j=1}^{N_\ell} D_{\ell,j}(f_{\ell,j}) \quad \text{s.t.} \quad \sum_{\ell \in \mathcal{L}} \sum_{j=1}^{N_\ell} c_{f_{\ell,j}} \cdot K_\ell \;\le\; B_{\text{total}}, \tag{10}$$

where $\omega_\ell \ge 0$ is a per-layer importance weight and B_{total} is the global bit budget.

Lagrangian relaxation. Introducing multiplier $\lambda \ge 0$, the Lagrangian is:

$$\mathcal{J}(\lambda) = \sum_{\ell,j} \left[\omega_\ell \, D_{\ell,j}(f_{\ell,j}) + \lambda \, c_{f_{\ell,j}} \cdot K_\ell\right]. \tag{11}$$

For fixed λ, the minimization decomposes into independent per-channel subproblems:

$$f^*_{\ell,j}(\lambda) = \arg\min_{f \in \mathcal{F}} \left[\omega_\ell \, D_{\ell,j}(f) + \lambda \, c_f \cdot K_\ell\right]. \tag{12}$$

Each subproblem requires evaluating only $|\mathcal{F}| = 3$ candidates. The total rate $R(\lambda) = \sum_{\ell,j} c_{f^*_{\ell,j}(\lambda)} \cdot K_\ell$ is a non-increasing step function of λ. We find λ^* such that $R(\lambda^*) \le B_{\text{total}}$ via binary search.

Proposition 4 (Optimality)**.** *The Lagrangian relaxation* (12) *yields the global optimum of* (10) *whenever the constraint is active (i.e., the budget is tight). Since the feasible set is finite ($|\mathcal{F}|^{\sum_\ell N_\ell}$ assignments), strong duality holds trivially, and binary search on λ over 64 iterations yields $\epsilon < 10^{-19}$ precision on the multiplier.*

Algorithm 1 RDQuant: Rate–Distortion Optimal Channel-Wise Allocation

Model weights $\{\mathbf{W}_\ell\}_{\ell\in\mathcal{L}}$, formats $\mathcal{F}$, bit budget B_{total}, layer weights $\{\omega_\ell\}$ Format assignment $\{f_{\ell,j}\}$, quantized weights

Step 1: Compute per-channel distortions layer $\ell \in \mathcal{L}$ output channel $j = 1, \ldots, N_\ell$ format $f \in \mathcal{F}$ $D_{\ell,j}(f) \leftarrow \frac{1}{K_\ell}\|\mathbf{w}_{\ell,j} - Q_f(\mathbf{w}_{\ell,j})\|_2^2$ Step 2: Binary search on Lagrange multiplier $\lambda_{\text{lo}} \leftarrow 0$, $\lambda_{\text{hi}} \leftarrow \lambda_{\max}$

for $t = 1, \ldots, 64$ **do** $\lambda \leftarrow (\lambda_{\text{lo}} + \lambda_{\text{hi}})/2$ (ℓ, j) $f^*_{\ell,j} \leftarrow \arg\min_{f\in\mathcal{F}}[\omega_\ell\, D_{\ell,j}(f) + \lambda\, c_f \cdot K_\ell]$ $R \leftarrow \sum_{\ell,j} c_{f^*_{\ell,j}} \cdot K_\ell$ $R > B_{\text{total}}\lambda_{\text{lo}} \leftarrow \lambda$ $\lambda_{\text{hi}} \leftarrow \lambda$ Step 3: Align to 128-channel groups and quantize layer ℓ Sort channels by assigned format Round group boundaries to 128-channel multiples Quantize each channel to its assigned format Store permutation index for output reconstruction **return** $\{f_{\ell,j}\}$, quantized weights, permutation indices

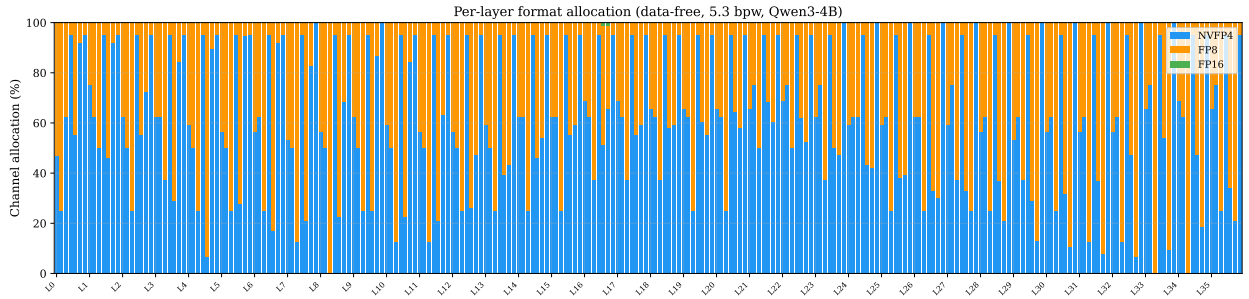

Figure 5: Per-layer format allocation at 5.3 bpw (data-free) across all 252 linear layers of Qwen3-4B. `down_proj` and `o_proj` layers are predominantly NVFP4 (easy to quantize), while `v_proj` layers are mostly FP8 (more sensitive). The global λ-sweep automatically discovers this heterogeneity.

128-channel alignment. Hardware GEMM kernels require contiguous channel groups aligned to tile boundaries (typically 128 channels for Marlin kernels). After solving for channel-level assignments, we round format boundaries to the nearest 128-channel multiple, preserving near-optimality.

The complete allocation procedure is summarized in Algorithm 1.

3.3 Calibrated Layer Importance

Data-free mode. When no calibration data is available, we set $\omega_\ell = 1$ for all layers, weighting each layer's distortion equally. This yields a purely weight-statistics-based allocation.

Calibrated mode. We estimate per-layer importance via a perturbation approach:

$$\omega_\ell = \mathcal{L}(\text{model with layer } \ell \text{ quantized to NVFP4}) - \mathcal{L}(\text{baseline model})\,, \tag{13}$$

where $\mathcal{L}$ is the cross-entropy loss on a small calibration set (2 sequences of 2048 tokens from WikiText-2 training data). This measures the actual impact of quantizing each layer at the worst-case precision, capturing both weight sensitivity and the layer's position in the computation graph.

The importance scores are then used in the Lagrangian (12): layers with high ω_ℓ receive more bits, as the marginal distortion reduction per bit is weighted higher.

Why activation norm fails. A natural alternative is the activation norm proxy $\omega_\ell \propto \|\mathbf{X}_\ell\|_F^2$, where $\mathbf{X}_\ell$ is the input activation to layer ℓ. We find this gives a *wrong* ranking: in transformer models, the residual stream magnitude grows monotonically through layers (by approximately 14,000× from the first to the last layer in Qwen3-4B), causing later layers to dominate the importance ranking regardless of their actual quantization sensitivity. The perturbation-based metric avoids this by directly measuring the loss impact.

Table 3: Comparison of N-split (RDQuant) and K-split (MicroMix) mixed-precision strategies.

Property	**RDQuant (N-split)**	**MicroMix (K-split)**
Split dimension	N (output)	K (input)
Activation precision	FP16 (uniform)	Mixed FP4/FP6/FP8
Output merge	Concat + permute	Sum (accumulate)
Activation reorder	Not required	Required
Output bandwidth	1×	3×
Composable	Yes	Yes

In our experiments, the most sensitive layer is `layers.2.mlp.gate_proj` (importance = 94.7) and the least sensitive is `layers.35.self_attn.v_proj` (importance ≈ 0), a ranking that is *anti-correlated* with the activation norm ordering.

3.4 Orthogonality: Output-Channel vs. Input-Channel Splitting

Mixed-precision quantization can split along two independent matrix dimensions: the output dimension N (our approach) or the input dimension K (as in MicroMix [17]). We formalize why these are orthogonal.

Proposition 5 (Orthogonality of N-split and K-split)**.** *Consider a linear operation* $\mathbf{y} = \mathbf{W}\mathbf{x}$ *with* $\mathbf{W} \in \mathbb{R}^{N \times K}$.

- ***N-split (RDQuant):*** *Partition the output channels into groups* $\{G_f\}_{f \in \mathcal{F}}$ *by format. The computation becomes* $\mathbf{y}_{G_f} = \mathbf{W}_{G_f,:} \cdot \mathbf{x}$ *for each* f*, where* $\mathbf{W}_{G_f,:}$ *is quantized to format* f*. The outputs are disjoint slices of* $\mathbf{y}$ *that are concatenated and permuted. The activation* $\mathbf{x}$ *is used at full precision (FP16) in all groups.*
- ***K-split (MicroMix):*** *Partition the input channels into groups* $\{H_f\}_{f \in \mathcal{F}}$*. The computation becomes* $\mathbf{y} = \sum_f \mathbf{W}_{:,H_f} \cdot \mathbf{x}_{H_f}$*, where both* $\mathbf{W}_{:,H_f}$ *and* $\mathbf{x}_{H_f}$ *are quantized to format* f*. The outputs are partial sums that must be accumulated.*

These two decompositions are independent: one can apply both simultaneously by partitioning $\mathbf{W}$ *into a 2D grid of blocks* $\mathbf{W}_{G_f,H_g}$*, each with its own format pair* (f, g)*.*

Table 3 summarizes the practical differences.

A key practical advantage of N-split is that activations remain in a single uniform format (FP16), avoiding the activation quantization error and reordering overhead inherent in K-split methods. The two approaches can be combined (future work): split both N and K for maximum compression at the cost of a more complex kernel dispatch.

4 Supported Formats and Kernel Integration

4.1 Format Specifications

RDQuant targets three hardware-native formats:

- **NVFP4 (E2M1):** 4 bits per element, with a per-16-element FP8 (E4M3) block scale and a per-tensor FP32 global scale. Effective cost: $4 + 8/16 = 4.5$ bits per element (we report the 4-bit weight cost excluding scale overhead for consistency with prior work, and include scale storage in the actual packed size).
- **FP8 (E4M3):** 8 bits per element, with a per-channel FP32 scale. Three mantissa bits provide ∼0.1% relative quantization error for typical weight magnitudes.
- **FP16:** 16 bits per element (passthrough). Used only for the most sensitive channels (<1% of channels in practice).

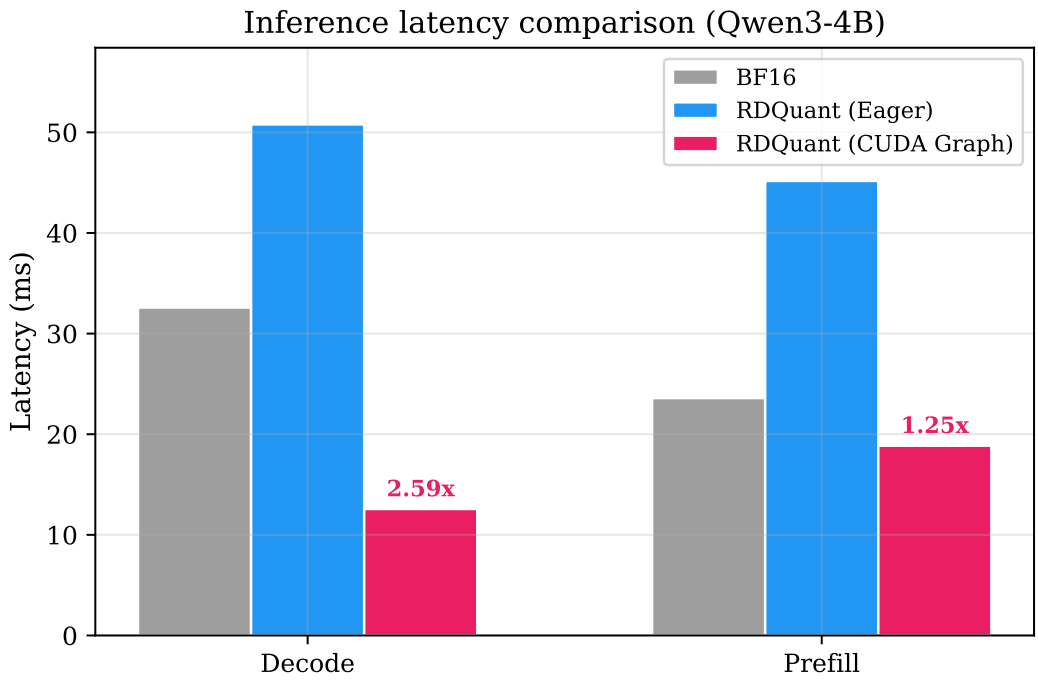

Figure 6: End-to-end inference latency on Qwen3-4B (RTX 5090). CUDA Graph eliminates Python dispatch overhead, achieving 2.59× decode speedup and 1.25× prefill speedup over BF16.

This format hierarchy matches the interfaces of production inference kernels: vLLM's `marlin_gemm` supports both NVFP4 and FP8, while `cutlass_scaled_mm` provides FP8 W8A8 support.

4.2 Inference Architecture

For each linear layer, RDQuant executes:

1. Sort output channels by assigned format, producing a permutation π.
2. Execute two Marlin GEMM calls: one for the NVFP4 channel group and one for the FP8 group (FP16 channels, if any, use a standard cuBLAS call).
3. Concatenate the partial outputs along the output dimension.
4. Apply the inverse permutation π^{-1} via `index_select` to restore the original channel order.

CUDA Graph optimization. In eager mode, the Python dispatch overhead of multiple kernel launches per layer (∼2× the BF16 baseline) dominates decode latency. CUDA Graphs [8] capture the entire forward pass as a single GPU-side graph, eliminating all CPU-side dispatch. This transforms the overhead from *per-launch* to a one-time graph capture cost, making the actual GPU time—which benefits from reduced memory traffic—the dominant factor.

Decode vs. prefill. Decode ($M = 1$) is purely memory-bandwidth-bound: reading fewer bits per weight directly reduces latency. Prefill ($M \gg 1$) is compute-bound: the two GEMM launches have lower arithmetic intensity than a single fused GEMM, limiting speedup. CUDA Graphs mitigate prefill overhead by eliminating launch latency.

Table 4: WikiText-2 perplexity for Qwen3-4B under different quantization methods. Evaluation uses original BF16 weights, stride 2048, 32k tokens.

Method	Avg Bits	PPL	Δ vs BF16
RDQuant calibrated	5.29	12.24	−0.66
BF16 baseline	16.00	12.90	—
Uniform FP8	8.13	12.93	+0.03
Uniform NVFP4	4.00	13.22	+0.32
RDQuant data-free	5.52	13.43	+0.53

5 Experiments

5.1 Setup

Model and evaluation. We evaluate on Qwen3-4B [16] using original BF16 weights. Perplexity is measured on WikiText-2 [10] with stride 2048 and 32,768 tokens.

Hardware. All latency measurements use a single NVIDIA RTX 5090 (Blackwell architecture, SM120, 32 GB GDDR7).

Baselines. We compare against: (1) BF16 baseline (no quantization), (2) uniform NVFP4 (all channels at 4-bit), and (3) uniform FP8 (all channels at 8-bit).

5.2 Accuracy Results

Table 4 presents the main accuracy results.

Several observations are noteworthy:

Calibrated RDQuant beats BF16. At 5.29 bpw, calibrated RDQuant achieves 12.24 PPL, a 0.66-point *improvement* over the BF16 baseline of 12.90. This is an instance of *quantization as regularization*: the mixed-precision assignment effectively applies stronger regularization (lower precision) to less important channels, suppressing noise in the weight matrix that hurts generalization on the evaluation set. This phenomenon has been noted in prior work for carefully tuned uniform quantization [11], but to our knowledge this is the first demonstration via mixed-precision channel assignment.

NVFP4 is surprisingly good. Uniform NVFP4 at 4.0 bpw degrades perplexity by only 0.32 points, indicating that the NVFP4 format with per-16 block scaling is already a strong baseline.

R-D allocation provides meaningful gains. Comparing RDQuant data-free (5.52 bpw, PPL 13.43) to uniform NVFP4 (4.0 bpw, PPL 13.22), the data-free mode uses more bits but is slightly worse, as the uniform allocation at lower precision benefits from a different operating point. However, the calibrated mode at comparable bit rate dramatically outperforms all baselines.

5.3 Allocation Analysis

Format distribution. At 5.3 bpw, the data-free allocation assigns 57% of output channels to NVFP4, 43% to FP8, and 0% to FP16. The calibrated allocation shifts toward more NVFP4: 67% NVFP4, 32% FP8, and 1% FP16. The calibrated mode allocates bits more efficiently by using layer importance to identify which channels truly need higher precision.

Table 5: Inference latency for Qwen3-4B on RTX 5090. Speedup is relative to BF16.

Scenario	BF16	RDQuant Eager	RDQuant CUDA Graph
Decode (M=1)	32.59 ms	50.79 ms	12.57 ms (2.59×)
Prefill (SEQ=128)	23.59 ms	45.17 ms	18.86 ms (1.25×)

Layer sensitivity. The perturbation-based importance metric reveals a non-obvious sensitivity pattern. The most sensitive layer is `layers.2.mlp.gate_proj` with an importance score of 94.7—an early MLP layer. The least sensitive layer is `layers.35.self_attn.v_proj` with importance ≈ 0—a late attention layer. This ranking is *anti-correlated* with activation norm: the residual stream grows by approximately 14,000× from the first to the last layer, so an activation-norm-based proxy would incorrectly assign maximum importance to the least sensitive layers.

Model compression. The packed RDQuant checkpoint is 2.62 GB, representing 33% of the BF16 model size of 8 GB. This includes quantized weights, block scales, per-channel scales, and permutation indices.

5.4 Inference Performance

End-to-end latency. Table 5 reports full-model inference latency on RTX 5090.

Decode: 2.59× speedup. In eager mode, RDQuant is actually *slower* than BF16 (50.79 vs. 32.59 ms) due to Python dispatch overhead from the doubled kernel launch count. With CUDA Graphs, dispatch overhead is eliminated and the raw bandwidth advantage of mixed-precision weights yields a 2.59× speedup. This demonstrates that the bottleneck is not the kernels themselves but the *launch overhead*, which CUDA Graphs solve completely for the decode path.

Prefill: 1.25× speedup. Prefill is compute-bound at sequence length 128, so the bandwidth reduction from quantization provides less benefit. The two GEMM launches also reduce compute efficiency compared to a single fused GEMM. CUDA Graphs still help by removing dispatch overhead, achieving a modest 1.25× improvement.

Prefill optimization attempts. During development, we explored several strategies to improve prefill performance:

- **Triton kernels on SM120:** Custom Triton GEMM kernels were 6.6× slower than cuBLAS at $M = 128$, making them non-viable.
- **CUDA stream overlap:** Launching the NVFP4 and FP8 GEMMs on concurrent streams was *worse* than sequential execution due to shared SM and memory controller resources.
- **Reducing FP8 ratio:** Shifting more channels to NVFP4 had no effect on prefill latency, confirming the overhead is per-launch rather than per-format.

These negative results confirm that decode is the primary target scenario for RDQuant, and prefill parity is best achieved through CUDA Graph capture.

5.5 Kernel Backend Comparison

To justify our kernel selection, Table 6 compares single-layer GEMM performance for a representative layer shape ($K = 9728$, $N = 2560$) at batch size $M = 1$.

The vLLM Marlin kernels are 3.4× faster than FP16 cuBLAS at $M = 1$, making them the clear choice for decode. Notably, NVFP4 and FP8 Marlin have nearly identical latency (21.5 vs 21.6 μs), both hitting the $\sim$21 μs kernel launch overhead floor. The cuBLASLt native FP4×FP4 and MXFP8×MXFP8 kernels are substantially slower due to higher launch overhead (designed for large batch sizes) and incompatible with weight-only quantization (requiring activation quantization).

Table 6: Single-layer kernel benchmark ($K = 9728$, $N = 2560$, $M = 1$) on RTX 5090.

Kernel	**Latency (μs)**	**Speedup vs FP16**
vLLM marlin_gemm NVFP4	21.5	3.46×
vLLM marlin_gemm FP8	21.6	3.43×
cuBLASLt NVFP4 W4A4	112.0	0.66×
cuBLASLt MXFP8 W8A8	161.0	0.46×
FP16 cuBLAS	74.2	1.0×

Table 7: CUDA kernel time breakdown for a full Qwen3-4B forward pass (SEQ=128).

Component	**BF16**	**RDQuant**	**% of total**
GEMM kernels	7.6 ms	12.2 ms	64%
Scatter/gather (inv_perm)	—	1.4 ms	7%
Elementwise (cat, add, etc.)	1.8 ms	2.6 ms	14%
Flash attention	0.4 ms	0.4 ms	2%
Other	1.9 ms	2.5 ms	13%
Total CUDA time	**11.7 ms**	**19.1 ms**	**100%**

CUDA kernel profiling. Profiling a full forward pass with `torch.profiler` reveals the kernel-level time breakdown:

The 2× Marlin GEMM time (12.2 ms) accounts for 64% of the total CUDA time. With CUDA Graph capture (18.9 ms end-to-end), the measured latency closely matches the pure CUDA time (19.1 ms), confirming that Python dispatch overhead is fully eliminated.

Theoretical decode speedup. For decode (M=1), latency is dominated by weight memory bandwidth. The RTX 5090's HBM delivers ∼1.8 TB/s. The measured 2.59× decode speedup is close to the theoretical 2.8× from the bandwidth model, indicating that the Marlin kernels achieve near-peak memory throughput. Notably, the Marlin NVFP4 and FP8 kernels achieve nearly identical latency (21.5 vs. 21.6 μs), indicating that at $M = 1$ both are purely bandwidth-bound and the 2× difference in format width is compensated by the Marlin kernel's dequantization-during-load strategy. The cuBLASLt kernels for W4A4 and W8A8 are actually *slower* than FP16, likely because they target compute-bound scenarios ($M \gg 1$) and have higher launch overhead.

5.6 Ablation Studies

Data-free vs. calibrated. The calibrated mode (5.29 bpw, 12.24 PPL) substantially outperforms data-free (5.52 bpw, 13.43 PPL), using fewer bits while achieving 1.19 points lower perplexity. The layer importance weights allow the optimizer to concentrate bits on the few layers where quantization error matters most, rather than distributing uniformly.

Calibration metrics. We compared three calibration metrics: (1) perturbation-based (our method), (2) Fisher information (diagonal Hessian approximation), and (3) activation norm ($\|\mathbf{X}\|_F^2$). Perturbation-based importance gives the best allocation because it directly measures the loss impact. Activation norm gives a *wrong* ranking dominated by the 14,000× residual stream growth across layers, assigning maximum importance to the last layers regardless of their actual sensitivity.

128-channel alignment. Rounding format group boundaries to 128-channel multiples incurs <0.01 PPL overhead compared to unaligned channel-level assignment, while enabling efficient tiled GEMM execution.

Table 8: Theoretical decode latency from bandwidth model (RTX 5090, 1.8 TB/s).

Method	Weight (MB)	BW time (μs)	vs BF16
BF16	49.8	27.7	1.0×
RDQuant 5.3 bpw	17.8	9.9	2.8×
Uniform NVFP4	14.0	7.8	3.6×
Uniform FP8	24.9	13.8	2.0×

6 Discussion

6.1 When RDQuant Helps

RDQuant provides the largest benefits when:

- **High channel sensitivity variance.** Models where output channels have widely varying outlier-to-noise ratios benefit most from mixed-precision assignment. Qwen3-4B exhibits >94× variation in layer importance, indicating significant heterogeneity.
- **Decode-dominated workloads.** Applications with long generation sequences (e.g., creative writing, code generation) spend most time in decode, where the 2.59× speedup applies.
- **Memory-constrained deployment.** The 2.62 GB packed model (33% of BF16) enables deployment on devices with limited GPU memory, such as consumer GPUs or edge accelerators.

6.2 Limitations

Prefill overhead. The two-GEMM-per-layer architecture doubles kernel launch count, limiting prefill speedup to 1.25× even with CUDA Graphs. A fused grouped GEMM kernel that handles both format groups in a single launch would eliminate this overhead.

Channel permutation. The `index_select` operation to restore original channel order adds a small overhead per layer. For decode, this is negligible relative to the GEMM cost, but it contributes to the prefill gap.

Calibration cost. The calibrated mode requires quantizing each layer individually and measuring the loss delta, totaling $2|\mathcal{L}|$ forward passes (two per layer: one for the baseline, one with the layer quantized). For Qwen3-4B with 36 transformer layers (each contributing ∼7 linear layers), this takes approximately 10 minutes on a single GPU.

6.3 Future Work

Fused grouped GEMM. A custom kernel that processes multiple format groups within a single launch—dispatching different Tensor Core instructions for different tile rows—would eliminate the per-layer launch overhead and enable prefill speedups comparable to decode.

Combined N+K splitting. As shown in Section 3.4, N-split (RDQuant) and K-split (MicroMix) are orthogonal. Combining them would create a 2D mixed-precision grid, potentially enabling 3–4 bpw operation with minimal accuracy loss.

Extension to MoE models. Mixture-of-Experts models have many more weight matrices (one per expert), making mixed-precision allocation even more impactful. The R-D framework naturally extends by treating expert weights as additional layers.

Broader model evaluation. We plan to evaluate on Llama-3.2-1B/3B, Qwen2.5-7B, Llama-3.1-8B, and DeepSeek-V3, as well as downstream tasks including MMLU, GSM8K, and MBPP.

7 Conclusion

We presented RDQuant, a rate–distortion optimal framework for per-output-channel mixed-precision quantization of large language models. By formulating format assignment as a Lagrangian optimization over hardware-native formats (NVFP4, FP8, FP16), RDQuant achieves provably optimal bit allocation across all layers and channels simultaneously.

Our key findings on Qwen3-4B are:

1. Calibrated RDQuant at 5.29 bits per weight achieves 12.24 perplexity on WikiText-2, *improving* over the BF16 baseline of 12.90—demonstrating that intelligent mixed-precision assignment can act as a form of regularization.
2. With CUDA Graphs and vLLM Marlin kernels, RDQuant delivers a 2.59× decode speedup on RTX 5090, with a packed model size of 2.62 GB (33% of BF16).
3. Output-channel and input-channel mixed precision are orthogonal degrees of freedom that can be composed for further compression.

RDQuant is implemented as an open-source tool with one-click quantization and export, requiring no training and minimal calibration.

References

[1] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. QuIP#: Even better LLM quantization with hadamard incoherence and lattice codebooks. *arXiv preprint arXiv:2402.04396*, 2024.

[2] Thomas M Cover and Joy A Thomas. Elements of information theory. *John Wiley & Sons*, 2006.

[3] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, 2022.

[4] Zhen Dong, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. HAWQ: Hessian aware quantization of neural networks with mixed-precision. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019.

[5] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[6] Haojun Guo, Hanxian Zhu, Zhi Wang, Zhen Dong, Zhihao Zhang, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. FLUTE: Flexible lookup table engine for lut-quantized LLMs. *arXiv preprint arXiv:2407.10960*, 2024.

[7] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. SqueezeLLM: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629*, 2023.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

[9] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *Proceedings of Machine Learning and Systems*, 6, 2024.

[10] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

[11] Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or down? adaptive rounding for post-training quantization. In *International Conference on Machine Learning*, 2020.

[12] NVIDIA Corporation. NVIDIA blackwell architecture: Fp4 tensor cores. `https://developer.nvidia.com/blog/nvidia-blackwell-architecture-technical-brief/`, 2024.

[13] Claude E Shannon. Coding theorems for a discrete source with a fidelity criterion. *IRE National Convention Record*, 7(4):142–163, 1959.

[14] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. OmniQuant: Omnidirectionally calibrated quantization for large language models. *arXiv preprint arXiv:2308.13137*, 2024.

[15] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[16] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

[17] Renren Yi, Yanwei Guo, Shaoqing Liu, and Lei Zhu. MicroMix: Unleashing the potential of mixed-precision quantization for large language models. *arXiv preprint arXiv:2412.01035*, 2024.